Table Variables vs Temp Tables

#sql-server #performance #optimizer #statistics

The Core Problem

Table variables (`Declare @T Table (...)`) have no statistics. The SQL Server optimizer always estimates that a table variable contains 1 row, regardless of how many rows you actually insert. This is hardcoded behavior in all versions through SQL Server 2017. (SQL Server 2019 introduced deferred table variable compilation, but only under compatibility level 150 and higher.)

When the optimizer thinks a table has 1 row, it makes decisions that are catastrophic for larger datasets. It chooses nested loop joins (perfect for 1 row, disastrous for 10,000), skips parallelism, and underestimates memory grants (which causes spills to tempdb).

Table Variable Behavior

```sql
Declare @ToteGroups Table (ToteId varchar(24), GroupNum varchar(10))

-- Insert 500 rows
Insert Into @ToteGroups
Select ToteId, GroupNum From SomeSource

-- Optimizer STILL thinks @ToteGroups has 1 row.
-- Nested loop join against a million-row table is a disaster.
Select G.*, O.*
From @ToteGroups G
Join OeBigTable O On O.GroupNum = G.GroupNum
```

What you'll see in the execution plan: the estimated rows on the table variable scan will say 1. The actual rows will say 500. Every downstream operator inherits this bad estimate, producing a cascading cardinality misestimate throughout the plan.

Temp Table Behavior

```sql
Create Table #ToteGroups (ToteId varchar(24), GroupNum varchar(10))

-- Insert 500 rows
Insert Into #ToteGroups
Select ToteId, GroupNum From SomeSource

-- Create an index for efficient joining
Create Index IX_ToteGroups_GroupNum On #ToteGroups(GroupNum)

-- Optimizer knows #ToteGroups has ~500 rows.
-- Chooses hash match or merge join, appropriate for the actual data.
Select G.*, O.*
From #ToteGroups G
Join OeBigTable O On O.GroupNum = G.GroupNum
```

Why temp tables work: SQL Server creates auto-statistics on temp tables. After the INSERT, the statistics reflect the actual row count and data distribution. The optimizer reads these statistics and makes informed decisions.

The Three Advantages of Temp Tables

1. Statistics (Cardinality Estimation)

The optimizer knows the real row count. This is the primary advantage and the one that matters most.

2. Indexes

You can create indexes on temp tables. Table variables support only primary key and unique constraints declared inline. You can't add non-clustered indexes after the fact. With a temp table, you can:

```
Create Table #ToteGroups (
    ToteId varchar(24),
    GroupNum varchar(10),
    ToteGroupNum tinyint
)

-- Insert data first, then create index.
-- This is actually faster than inserting into an indexed table.
Create Index IX_ToteGroups_GroupNum On #ToteGroups(GroupNum)
```

The "insert then index" pattern is a best practice. Building the index on the complete dataset is a single efficient sort, versus maintaining the b-tree during every insert.

3. Parallelism

Table variables block parallelism in certain plan shapes. Temp tables never do.

When Table Variables Are Fine

Table variables aren't always bad. They're appropriate when:

- The data is always small (under 100 rows, guaranteed by business logic).
- You're using them as simple accumulators (single-row OUTPUT targets, etc.).
- You need transaction isolation. Table variable data is not affected by ROLLBACK. This is occasionally useful for audit logging.

The Migration Pattern

When refactoring from table variable to temp table:

```
-- BEFORE
Declare @Results Table (Id int, Name varchar(100))
Insert Into @Results Select Id, Name From Source Where ...
Select * From @Results R Join BigTable B On B.Id = R.Id

-- AFTER
Create Table #Results (Id int, Name varchar(100))
Insert Into #Results Select Id, Name From Source Where ...
Create Index IX_Results_Id On #Results(Id)
Select * From #Results R Join BigTable B On B.Id = R.Id
```

Don't forget cleanup. Temp tables persist for the session (or scope of the proc), but explicit `Drop Table #Results` at the end is good hygiene.

Real-World Example

In lsp_RxfGetListOfManualFillGroups (v48 to v49), the `@ToteGroups` table variable was populated with tote/group mappings and then joined against multiple large tables. With the 1-row estimate, the optimizer chose nested loops for every join. Converting to `#ToteGroups` with an index on `GroupNum` allowed the optimizer to see the actual row count and choose appropriate join strategies, contributing to a 67% reduction in total logical reads.

Related Concepts

- Parameter Sniffing: another statistics and estimation problem.
- Density Vector: the statistics the optimizer uses for temp tables.
- Correlated Subqueries to CTEs: another pattern that helps the optimizer see set-based row counts.